

TD 1 : UNIX et Shell

Remarques: Les deux premiers TP du cours Sécurité Administration Système font à la fois office d'introduction, et de révision des connaissances de base du système d'exploitation GNU/Linux que vous utiliserez tout au long de votre cursus. Toutes les notions abordées ici seront considérées comme acquises et maîtrisées pour tous les cours à venir quelque soit le module.

Ce premier TP vous invite à (re)découvrir la ligne de commande (shell).

La connexion à votre session vous donne accès à votre environnement de travail. Vous choisirez un interpréteur de commandes (Terminal) pour saisir les instructions de ce TP.

1 Ligne de commande : mode interactif

Dans cette première partie, nous utiliserons la ligne de commande en mode interactif, à chaque commande correspond un retour plus ou moins immédiat.

1.1 Terminologie

Afin de se comprendre lorsque l'on parle du shell, définissons un certain nombre d'éléments :

— interpréteur de commande :

— terminal :

— commande :

— shell :

— prompt :

— invite de commande :

— home / homedirectory :

— root :

— utilisateur :

— login :

— racine :

— complétion :

— wildcard :

De manière générale, et en très simplifié, les commandes sont saisies par l'utilisateur dans le terminal où elles sont «buffurisées». L'utilisateur «valide» sa commande, ou suite d'instructions, par un retour chariot (touche «Enter» ou «Entrée»). Le contenu du buffer est alors transmis à l'interpréteur de commande pour exécution.

1.2 Trouver de l'aide

Lorsque l'on recherche de l'aide sur l'utilisation d'un programme, le réflexe est d'ouvrir son navigateur internet et de lancer une recherche sur son moteur de recherche favori. Cependant, il est parfois plus rapide de consulter la documentation locale. Sous Linux, elle est accessible depuis le shell via la commande **man** (pour manuel).

1.2.1 Man pages

Lancez la commande : **man man** et lisez le document. De quelle commande parlent ces pages ? A quoi sert cette commande ?

1.2.2 Utilisation des pages de manuel

Toutes les pages de **man** sont organisées de la même manière.

1. NAME : nom de la commande ;
2. SYNOPSIS : résumé court de la syntaxe de la commande ;
3. DESCRIPTION : description longue ou courte de la commande ;
4. OPTIONS (facultative) : description des options supportées ;
5. Il existe bien sûr la possibilité de mettre autant de sous-sections que l'on désire ;
6. AUTHOR : les personnes qui ont développé la commande ;
7. SEE ALSO : des références croisées.

1.2.3 Manuel de la commande ls

Vous allez maintenant lire la page d'aide et tenter d'y trouver des informations répondant aux questions suivantes :

1. En 10 mots maximum, à quoi sert cette commande ?
2. Quelle option permet d'afficher tous les fichiers (même ceux cachés : ceux dont le nom commence par un point) ?
3. Quelle option permet d'afficher les fichiers avec un format long en liste ?
4. Quelle option permet d'afficher la taille des fichiers sous une forme facilement lisible par l'homme ?
5. Quelle option permet d'afficher récursivement le contenu d'un répertoire ?

Utilisez le «joker» «*» pour lister tous les fichiers commençant par «u» à la racine de votre système.

1.2.4 La commande cd

- A quoi sert cette commande ?
- Rendez-vous Dans le répertoire **tmp** à la racine du système et lister son contenu
- retournez dans votre home directory
- Que fait **cd** - ?

1.2.5 Manuel de la commande mkdir

Considérons que nous voulons créer à la racine de votre compte le chemin suivant :

```
ArchiSys / tp / 01
```

Lancez la commande suivante et observez ce qui se passe :

```
cd ; mkdir ArchiSys / tp / 01
```

Cherchez dans la page de man de la commande «mkdir», l'option qui vous permet de créer une hiérarchie de répertoire en une seule commande.

1.3 La commande echo

à quoi sert la commande «echo» ?
Que fait la commande suivante ? Pourquoi ?

```
echo -ne "\n\n*\tHello_World_$LOGNAME\t\t*\n\n"
```

Essayez la commande suivante, quel est la différence ? Pourquoi ?

```
echo -e '\n\n*\tHello_World_$LOGNAME\t\t*\n\n'
```

1.4 Autres commandes

Dans la suite, nous utiliserons la commande «ps» qui liste les programmes en cours d'exécution sur la machine et «cat» qui lit une entrée et la restitue sur la sortie standard. Consultez les pages de manuel de ces commandes si elles ne vous sont pas familières.

2 Les redirections

Comme vous l'avez tous remarqué sans y prêter attention, l'exécution d'une commande en ligne de commande peut vous afficher des informations à l'écran. Par exemple «ls» vous liste le contenu du répertoire courant.

Par défaut, le résultat de l'exécution d'une commande est affiché sur la «**sortie standard**» (aussi appelée «**stdout**»). Il est possible de changer ce comportement par défaut pour «afficher» dans un fichier plutôt que sur le terminal le résultat d'une commande : cette pratique s'appelle la redirection.

Notez également que tout programme possède sa sortie standard, mais ne l'utilise pas forcément.

De manière générale, la plupart des programmes fonctionnent comme suit :

- si un ou plusieurs fichiers sont spécifiés, le programme travaille sur ces fichiers ;
- si aucun fichier n'est spécifié, l'entrée standard est utilisée.

2.1 Premières redirections

Exécutez chacune des commandes suivantes (dans l'ordre) dans votre terminal, examinez les résultats et déduisez ce qui se passe (il faudra bien évidemment regarder le contenu des fichiers générés) :

```
ls
ls > file1
pwd > file1
ps aux > file2.txt
cd > file3
cat file1 file2.txt > file4
cat file1 >> file1
pwd >> file1
cat file1 > /dev/null
```

à quoi servent les mots clefs «>» et «>>» dans les exemples précédents ?

2.2 Le pipe

Le pipe est aussi une forme de redirection, mais cette fois au lieu de rediriger la sortie standard sur un fichier comme précédemment, nous allons la rediriger vers une commande (pour simplifier, un programme). Notez que, comme pour la sortie standard, tout programme possède une «**entrée standard**», «**stdin**», qui peut ou non être utilisée. Dans le terminal, l'entrée standard est souvent le clavier.

2.2.1 pipe et cat

Lancez la commande cat dans un terminal. écrivez quelques mots à l'écran puis appuyez sur «Enter» (ou «Entrée» en fonction de votre clavier). La ligne est dupliquée. Pourquoi ?

2.2.2 Exemples de pipe

Exécutez chacune des commandes suivantes (dans l'ordre) dans votre terminal, examinez le contenu des fichiers manipulés et en déduire ce qui se passe :

```
ls -R > file1
less file1
cat file1
cat file1 | less
ps aux
ps aux | grep -v root
ps aux | grep root
ps aux > file2
cat file2 | grep -v root
```

Questions complémentaires : à quoi sert grep ? et son option '-v' ?

Lister les processus n'appartenant pas à «root» mais contenant root dans leurs noms et les consigner dans le fichier 'processwithroot'.

2.3 Redirection d'entrées

Tout comme il est possible de rediriger la sortie d'une commande, il est parfois intéressant de rediriger son entrée. Expliquez le comportement de la commande suivante :

```
cat <<eob
a
b
c
eob
```

3 Gestion de processus

Nous désignons par «gestion de processus» la manipulation des programmes en cours d'exécution à partir d'un terminal. Il existe bien sur des clients graphiques, comme sous Windows ou Mac OS, qui permettent de faire ce que nous allons faire dans cette section en ligne de commande.

Un programme en cours d'exécution s'appelle un **processus**. Les processus sont identifiés sur le système par un numéro appelé «PID» (process identifier).

Sous Unix en ligne de commande, un processus peut être exécuté de deux manières différentes :

- En tâche de fond (background) : lorsque le processus est lancé en tâche de fond, le terminal vous rend la main immédiatement, alors que le processus continue à s'exécuter : une commande qui s'exécute rapidement vous rendra la main immédiatement alors qu'elle n'est pas lancée en tâche de fond ;
- En avant-plan (foreground) : le terminal vous rend la main seulement quand le processus s'est terminé.

Nous allons voir qu'il est possible de basculer un processus d'un état à un autre dans le terminal, et de tuer des processus. Nous verrons aussi qu'il est possible de lister les processus en cours d'exécution dans un terminal grâce à la commande «**jobs**».

3.1 Processus en tâche de fond

Le «et commercial», aussi appelé «esperluette» en fin de commande permet de lancer un processus en tâche de fond :

```
xterm &  
emacs monfichier &
```

Dans ce cas, le Shell exécute la commande xterm et rends la main après avoir affiché le PID numéroté par le shell depuis 1, puis il lance de même emacs.

Exemple de retour après xterm :

```
[1] 1360
```

puis après le emacs :

```
[2] 1366
```

Dans ce cas, le PID d'xterm est «1360» et ce processus est numéroté «1» dans le shell tandis que emacs (PID 1366) est numéroté «2» dans ce shell.

3.2 Processus en avant-plan

Lancez les mêmes commandes que précédemment mais sans l'esperluette.

3.3 Basculement d'un état à un autre

Lorsqu'un processus est lancé en «foreground», vous ne pouvez plus interagir avec le Shell qui l'a lancé. Pour «reprenre la main» il faut basculer le processus en «background». La séquence de touche «CTRL + Z» met tout d'abord en sommeil le processus en cours d'exécution en avant-plan, ensuite la commande «bg» (pour «background» bien sûr) réveille le processus et l'exécute en tâche de fond.

NB : La touche «Contrôle» peut être notée «CTRL» ou «^». Ces notations sont équivalentes.

à l'inverse, le dernier processus lancé en background (avec le «&»), peut être basculé au premier plan avec la commande «fg» suivie de «Enter» (ou «Entrée» en fonction des claviers).

- Dans un terminal lancer un nouveau terminal «xterm» ;
- lancer la commande ls dans le nouveau terminal ;
- mettre le nouveau terminal en sommeil ;
- lancer la commande ls dans le nouveau terminal, que se passe-t-il ? Pourquoi ?
- basculer le terminal en arrière plan
- relancer «ls» que se passe-t-il ? Pourquoi ?

3.3.1 Application sur les états de processus

Saisir le code suivant dans le fichier "script1" (utiliser cat et une redirection) :

```
while true ; do  
for i in {1..5} ; do  
echo bla  
sleep 1  
done  
echo -n ">>_"  
read a  
echo $a  
done
```

Lancez la commande suivante «bash script1», lorsque la ligne de terminal affiche «>> » saisir un caractère et valider par «Enter» ou «Entrée».

Après plusieurs exécutions, mettre le script en sommeil. Que se passe-t-il ?

Réveiller le processus en avant-plan, que se passe-t-il ?

Basculer le processus en arrière-plan. Que ce passe-t-il si vous saisissez «ls» lors de l’affichage de «>>» ? Proposez une explication.

Reprenez la main sur le processus le script et arrêtez le.

3.4 Tuer un processus en foreground

Pour tuer un processus exécuté en premier plan, saisir la séquence «CTRL + C» dans le terminal exécutant le processus.

3.5 jobs

En vous aidant de la page d’aide du bash à la section «JOB CONTROL», exécutez les commandes ou séquences claviers suivantes et analysez le comportement des processus (vous aurez bien entendu besoin de deux terminaux, un pour la manpage, l’autre pour l’exercice).

NB : Pensez à vous repositionner dans le terminal après le lancement des applications.

```
emacs &  
xterm &  
firefox &  
emacs  
Ctrl + C  
jobs  
fg %3  
Ctrl + Z  
bg  
fg %2  
Ctrl + C  
kill %1  
jobs
```

Quel processus reste-t-il après l’exécution de ces commandes ?

3.6 Chaîner des processus

Lorsqu’un programme termine son exécution, il retourne une valeur liée à la manière dont le processus s’est arrêté. Cette valeur peut être utilisée afin de chaîner des commandes. Pour cela, nous utiliserons les opérateurs «&&» pour «ET» et «||» pour «OU».

3.6.1 Première chaîne de processus

Avec la commande «grep», testez l’existence du compte «root» sur votre système et afficher le message : «root existe» si le compte est trouvé.

NB : le fichier «/etc/passwd» contient la liste des comptes existant sur le système.

3.6.2 Chaîne plus évoluée

En utilisant ces opérateurs et ce qui a été vu précédemment, écrire une commande affichant **UNIQUEMENT** le texte «Firefox est en cours d’exécution» sur la sortie standard si le programme firefox est exécuté sur votre machine et «Pas de firefox actuellement» sinon.

4 Le système d’exploitation GNU/Linux

4.1 Généralités

- La racine du système d’exploitation Linux est notée "/".
- Sous Linux tout est fichier.

- Un utilisateur est une personne ou un programme enregistré sur le système (disposant d'un compte).
- Des groupes permettent de classer les utilisateurs en fonction de critères définis par l'administrateur du système.
- Les utilisateurs peuvent appartenir à un ou plusieurs groupes.
- Les utilisateurs disposent de privilèges en fonction de leurs groupes d'appartenance.
- Pour le système, un utilisateur est un numéro appelé «uid» (user identifier) associé à un «login».
- Pour le système, un groupe est également un numéro appelé «gid» (group identifier).
- L'utilisateur appartient par défaut à un groupe dont le numéro est un «gid».
- Les droits, ou permissions, pouvant être affectés à un fichier sont : aucun droit noté «-», «lecture» noté «r», «écriture» noté «w» et exécution ou droit de traverser noté «x».
- Les droits peuvent être octroyés aux utilisateurs, aux groupes ou aux autres (personnes n'appartenant pas aux deux ensembles précédents).
- Les droits sont affichés sous la forme de triplets "rwx" pour chaque ensemble auquel ils peuvent être octroyés (au final nous obtenons des permissions allant de «- - - - -», aucun droit, à «rwxrwxrwx» tous le monde peut accéder en lecture écriture au fichier).
- L'option «-l» de «ls» permet de consulter les droits appliqués aux fichiers.
- La commande «chmod» (change mode) permet de modifier les droits sur les fichiers.
- La commande «chown» (change owner) permet de modifier le propriétaire et/ou le groupe propriétaire d'un fichier.
- Sur un système d'entreprise, un utilisateur simple ne peut élever ses privilèges sans autorisation.
- Le répertoire personnel d'un utilisateur est noté «~login». Si «login» est omis, l'utilisateur courant est sous-entendu.
- «groups» Permet de lister les groupes d'appartenance d'un utilisateur et «id» précise leur numéro.
- «pwd» (print work directory) Vous localise sur le système (affiche le répertoire courant dans lequel vous vous trouvez).

4.2 Questions

- Quels sont les droits sur votre répertoire personnel ?
- A quel groupe appartient votre «homedir» ?
- A quels groupe appartenez-vous ? et votre voisin ?
- Rendez-vous dans le «homedir» de «fhh». Que se passe-t-il ? Pourquoi ?
- Quel est votre gid ?
- «chmod» Dispose d'une notation octale des droits. Donner deux commandes, une avec notation octale, l'autre avec notation alphabétique, permettant de définir les droits suivants sur le répertoire «mydir» (droits de départ «rwxr-xr-x») :
 - rwx- - - - -
 - rwxr-x- - -
- Changez le propriétaire du répertoire tmp. Que se passe-t-il ? Expliquez.

5 Les variables

Sous Unix, le système est capable d'échanger ou de récupérer des informations par l'utilisation de variables. Ces variables sont rendues accessibles par le Shell. Certaines sont des variables du système d'exploitation (variables d'environnement), d'autres sont propres au Shell. Nous allons en étudier quelques unes, comment les modifier, les créer ou les afficher.

5.1 Déclaration

La déclaration d'une variable est du type :

nomvar=valeur

Où «valeur» est une chaîne de caractères, et «nomvar» est une chaîne composée des caractères ASCII de base et ne commençant pas par des chiffres. Par exemple :

```
name=" Jean "
```

5.2 Affichage

Le bash (qui est un type de Shell) vous fournit la commande «echo». Cette commande vous permet d'afficher la valeur associée au nom d'une variable. Les variables doivent être préfixées par le caractère «\$». La commande suivante affiche la valeur associée à la variable dénommée «name» :

```
echo $name
```

Bien que cette notation fonctionne, la notation recommandée par bash est :

```
echo ${name}
```

5.3 Modification

Modifier une variable consiste à écraser sa précédente déclaration.

5.4 Exercices

1. Affichez le contenu de la variable HOME. A votre avis que représente cette variable ?
2. Affichez le contenu de la variable PWD. A votre avis que représente cette variable ?
3. Affichez le contenu de la variable PATH. A votre avis que représente cette variable ?
4. Affichez le contenu de la variable PS1. A votre avis que représente cette variable ? Cherchez dans la page d'aide du bash.
5. Modifiez le contenu de la variable PATH, puis essayer de lancer une commande. Que se passe t'il ?
6. Modifiez la variable PS1, lancez un nouveau terminal. Que constatez-vous ?
7. Modifiez le fichier de configuration du bash pour y inclure votre définition de la variable PS1. Quelles sont les différences de comportement ?

5.5 Chaîne de caractères

Avec le shell vous pourrez trouver des chaînes de caractères entre double quotes ("foo"), simple quote ('foo') ou anti-quote (`foo`). Testez les commandes suivantes :

```
echo $HOME
echo "$HOME"
echo '$HOME'
echo `HOME`
TEST=ls ; echo "$TEST"
test=$(ls) ; echo $test
```

6 Les scripts

Un script est un ensemble d'instructions consignées dans un fichier qui seront exécutées séquentiellement.

Dans la suite de ce TP nous utiliserons l'interpréteur de commande bash.

Tous les blocs d'instruction d'un script peuvent être testés directement dans le shell.


```
cat > myscript
cd /tmp
ls
cd
^d
```

Le script est exécuté depuis la ligne de commande :

```
bash myscript
```

Dans cet exemple nous précisons le shell à utiliser («bash») et lui passons en argument le fichier contenant les instructions.

Nous pouvons également préciser, directement dans le script, l'interpréteur de commande à utiliser en le spécifiant dans l'entête du fichier

```
cat > myscript
#!/usr/bin/bash

cd /tmp
ls
cd
^d
```

Cette première ligne s'appelle le «**shebang**».

La commande «whereis» permet de localiser le shell que vous souhaitez utiliser :

```
whereis bash
```

le script peut ensuite être invoqué comme n'importe quel programme.

```
./myscript
```

Là, ça ne fonctionne pas, pourquoi ? Corrigez.

6.1 Structure de contrôle

«man test» donne la liste des tests disponible en bash.

6.1.1 if .. then ... else

Syntaxe :

```
if commande(s) ; then
    commande(s)
[ elif commande(s) ; then
    commande(s) ] ...
[ else commande(s) ]
fi
```

En utilisant un «if», testez l'existence du répertoire «/var/logs», puis du répertoire «/var/IOg» et afficher un commentaire en cas de répertoire trouvé ou non.

NB : Dans le cas de ligne de commande longue, vous pouvez utiliser «\» pour passer à la ligne et continuer votre commande.

Afficher «Alerte» si vous avez le droit d'écrire dans «/etc».

6.1.2 for ... do ... done

Syntaxe :

```
for var in list ; do
  commande(s)
done
```

Ecrire un «for» qui affiche les lettres «a b c d» a raison d'une par ligne.

6.1.3 while ... do ... done

Syntaxe :

```
while commande(s) ; do
  commande(s)
done
```

Que fait la séquence suivante ?

```
ls | while read a ; do echo $a ; done
```

6.2 Script poubelle

écrire un petit script qui prend en paramètre un ensemble de fichiers et les déplace dans la poubelle (un répertoire «.trash» à la racine de votre répertoire utilisateur). Afficher un message d'erreur si un fichier n'existe pas. Dans un premier temps, trouvez la variable spéciale qui représente tous les paramètres de la ligne de commande (voir man bash). Réalisez le petit script (10 à 15 loc).

7 Gestion des droits

Nous verrons dans cette section principalement des révisions sur la gestion des droits. J'en profite pour vous rappeler que toutes les ressources UNIX (fichiers, cdrom, disques, sockets, cartes sons ...) sont vues comme des fichiers. Vous pourrez alors préciser des droits d'accès aussi bien sur un disque dur que sur n'importe quel fichier.

Nous allons donc étudier ici les commandes **chmod**, **chown** et **chgrp**.

7.1 Question

À quoi servent ces trois commandes ?

7.2 Question

Créez à la racine du système un répertoire **data**, et donnez les droits d'accès en lecture, écriture et exécution à tout le monde.

7.3 Question

Imaginez alors que l'utilisateur bob crée un fichier **toto.txt** dans le **/data**, avec les droits 700. L'utilisateur john pourra-t-il supprimer le fichier ?

7.4 Question

Quel problème cela soulève-t-il ?

7.5 Question

Ce problème peut-il être résolu ? Si oui, donnez la ou les commandes à effectuer.

8 Une petite touche de Sherlock Holmes

Vous n'avez encore aucune (in)formation sur comment faire des attaques. L'une des premières chose à faire est une annalyse par attaque d'inférence. L'inférence est le principe de poser des questions autorisées dans le but d'obtenir des informations interdites. Cela regroupe bien évidemment beaucoup de chose : scan réseau, analyse de répertoires, demandes de pages WEB inexsitantes, lancement de programmes, analyse des codes et des configurations des applications installées. Bref en résumé : faire un audit du système.

Dans l'ordre, nous pouvons nous interesser :

- aux fichiers de configuration dans /etc, infos matérielles dans /proc
- aux packages (logiciels) installés (liste et versions)
- aux points de montages
- à la echerche de programmes sensibles
- à la connectivité de la machine

Pour chaque point, tentez de voir sur le site cvedetails ([https ://www.cvedetails.com/](https://www.cvedetails.com/)) s'il existe des failles logicielles ou materielles concernant les informations que vous avez trouvées. Nous en parlerons plus en details en cours. Si vous trouvez une faille trouvé, vous pourrez tenter de déterminer son type et trouver son explication dans la base "[https ://cwe.mitre.org/](https://cwe.mitre.org/)" et s'il elle s'applique vraiment à votre cas (par ex. est-ce la bonne version du logiciel ?).

Pour chaque section, vous partagerez avec vos collègues et l'enseignants les informations collectées. A la fin du module, nous vous engageons a refaire cet exercice avec vos nouvelles compétences et voir si vous avez trouvez plus de chose.

8.1 Fichiers de configuration, points de montages et systèmes de fichiers spéciaux

Il existe des systèmes de fichiers virtuels sous linux. Ces SF représentent l'état du systèmes et pas des fichiers réelles. Les trois plus connus sont '/dev/', '/proc/', '/sys'. '/dev' contient tous les périphériques. '/proc' contient les processus et certaines informations matériels (cpu, ram). '/sys' contient essentiellement la configuration noyau.

8.1.1 Question

Essayez d'écrire dans /dev/null, /dev/mem, /dev/sda.

8.1.2 Questions

Determiner les périphériques, la mémoire, les cpus.

Essayer de voir s'il peut exister des problèmes d'espaces disques potentiels.

A quoi correspond le point de montage '/boot/efi' ?

8.1.3 Questions

Essayer de trouver la version et la distribution de l'OS installé sur vos machines.

Tenter de voir la configuration pour le changement d'identité (sudoer).

Chercher des informations sur le(s) compte(s) root.

Regarder comment sont démarrés les services. L'outil le plus utilisé actuellement sous linux pour gérer le démarrage des machines est systemd.

8.1.4 Question

Obtenir des informations sur les logs de la machine. Tenter de lire les logs.

8.1.5 Question

A faire avec prudence!!! Tentez de changer votre mot de passe. Et regarder si des conditions sont imposées sur les mots de passe du système (taille, complexité).

8.2 Setuid / Setgid. A la recherche de programmes sensibles

Ici on fait un petit rappel de système. En plus des permissions de lecture/écriture/exécution vues dans la section précédente, il existe des permissions UNIX permettant d'exécuter un fichier avec les droits particuliers.

8.3 Question

Avec quels droits (de processus), par défaut, les programmes sont-ils lancés (sous linux) ?

Quels sont les droits (de fichier) associés aux fichiers `/usr/bin/passwd` et `/usr/bin/gpasswd` ? Expliquer leur intérêt pour ces programmes ?

8.3.1 Question

Vérifier les droits (de processus) avec lesquelles sont lancés les programmes classiques des utilisateurs ?

Constater les droits spéciaux (de processus) avec lesquelles sont lancés les programmes décrits précédemment ?

Comment trouvez tous ces programmes sensibles qui n'ont pas le comportement par défaut et trouver les packages auxquels ils appartiennent ?

8.4 Question

écrire un programme en C permettant d'afficher les identifiants user et group effectif. (voir man 2 `getuid`)

8.5 Question

Assigner les bits de droits `suid` et `gid` à votre programme.

8.6 Question

Exécuter le programme avec l'un des utilisateurs créés dans la section précédente, que remarquez-vous ?

8.7 Question

écrire un script shell permettant d'afficher les identifiants user et group effectifs, lui assigner le bit `suid`, et l'exécuter avec un autre utilisateur. Que remarquez-vous ?

8.7.1 Questions

INTUITIVEMENT : Pourquoi les programmes '`suid`' sont-ils considérés comme sensibles et dangereux ? Que se passe-t-il si on arrivait à injecter du code '`anormal`' dans ces programmes ?

8.7.2 Question

Essayer de savoir si selinux /apparmor est déployé.

8.8 Package (logiciels) installés (liste et versions)

8.8.1 Questions

Essayer d'obtenir la liste de tous les package de la machine en cherchant à partir de la distribution le noms des outils de package.

Sélectionner des packages qui vous semblent interessant et tenter de voir les fichiers qui sont installés par celui ci.

Chercher dans la base CVEDETAILS d'éventuelles failles.

Essayer de savoir de quand date la dernière mise à jour du système ? L'administrateur de la machine est-il consciencieux ?

8.9 La connectivité de la machine

8.9.1 Questions : Les ports ouverts et les connexions

Utiliser «lsof» ou «netstat» ou «ss» pour otebnir la liste des ports en écoute (et déterminer leur typer UDP/TCP/autre).

Essayer de trouver à quels services ils correspondent. Vous pourrez a titre d'exemple obtenir des informations sur le service ssh.

Obtenir les routes de la machine.

8.9.2 Questions : Scanner la réseau

Ecrire deux boucle testant votre environnement réseau. En premiere boucle vous utiliserez «nslookup» ou «dig» pour obtenir la nom des machines associer aux IPs.

Essayer d'obtenir des informations sur le domaine auquel vous appartenez.

En seconde boucle vous utiliserez «ping» pour tenter d'appeler les 254 machines de votre plage IP et n'afficherez que les adresses IP disponibles (ayant répondu au ping).

8.9.3 Questions : La protection des machines

Tenter de connaître la configuration du firewall des machines.

Tenter d'ouvrir différentes connexions entre les machines.

9 Galerie photo

L'objectif de cette partie est de créer un script générant une galerie photo html.ec

Dans un répertoire se trouve des photos aux formats PNG et JPG. Ce répertoire peut contenir des sous-répertoires, correspondant aux collections de photos.

Nous souhaitons un script bash prenant en argument le répertoire contenant les photos et générant une page html contenant la liste des collections et une icône associée, puis une icône de chaque image classée par date de prise de la photo. Lorsque l'on clique sur une icône de photo, la photo apparaît en 1024x768 (en conservant le ratio longueur largeur). Les images sont pivotées en fonction de la prise de vue. Dans le répertoire contenant les photos, ou les collections, un fichier "comment.txt" permet de définir des commentaires au format "nom_photo : Commentaire sur la photo" qui sera affiché sous la photo lors de l'affichage plein écran.

Toutes les évolutions imaginées sont les bienvenues.

Pour extraire la position des photos et les dates de prises de vues, vous utilisez le programme «exiv2».

Les icônes des images appelées «thumbnail» seront générées via la commande «convert». Les icônes déjà générées ne seront pas régénérées (utilisez le md5sum des fichiers comme nom d'icône par exemple).